

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Double Linked List pada Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Program NodeDLL	2
2.2 Program InsertDLL	2
2.3 Program PenelusuranDLL	5
2.4 Program HapusDLL	6
BAB III TUGAS	10
3.1 Program Lagu.....	10
3.2 Program Musik.....	11
BAB IV KESIMPULAN	15
4.1 Ringkasan Hasil Praktikum	15
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Struktur Data Double Linked List pada Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman struktur data, Single Linked List (SLL) memiliki kelemahan utama yaitu penelusuran yang hanya bisa dilakukan dalam satu arah (maju). Untuk mengatasi limitasi ini, dikembangkanlah struktur data Double Linked List (DLL). Berbeda dengan SLL, setiap simpul (node) pada DLL dilengkapi dengan dua buah pointer, yaitu pointer next untuk menunjuk ke simpul setelahnya, dan pointer prev untuk menunjuk ke simpul sebelumnya.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami perbedaan fundamental antara Singly Linked List dan Doubly Linked List, khususnya pada manajemen dua buah penunjuk arah (pointer next dan prev).
2. Mengimplementasikan operasi dasar penambahan (insertion) dan penghapusan (deletion) elemen DLL pada berbagai posisi (awal, tengah, dan akhir).
3. Menerapkan operasi penelusuran (traversal) secara dua arah, baik dari head menuju tail (maju) maupun dari tail menuju head (mundur).

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Meningkatkan pemahaman mengenai manajemen memori dinamis dan pointer di Java.
2. Meningkatkan fondasi logika untuk membangun struktur data yang lebih kompleks di masa depan.

BAB 2

PEMBAHASAN

2.1 Program NodeDLL

```
package pekan6_2511532001;

public class NodeDLL_2511532001 {
    // mendefinisikan kelas node
    int data_2001;
    NodeDLL_2511532001 next_2001;
    NodeDLL_2511532001 prev_2001;

    // konstruktor
    public NodeDLL_2511532001(int data) {
        this.data_2001 = data;
        this.next_2001 = null;
        this.prev_2001 = null;
    }
}
```

Class NodeDLL merupakan representasi dari sebuah entitas simpul dalam struktur Double Linked List. Berbeda dengan simpul SLL yang hanya memiliki satu penunjuk, kelas ini secara spesifik mendeklarasikan tiga buah atribut, data untuk menyimpan nilai integer, next untuk menyimpan referensi alamat memori simpul berikutnya, dan prev untuk menyimpan alamat memori simpul sebelumnya. Pada saat objek node baru diinisialisasi melalui konstruktor, pointer referensi tersebut akan di-set bernilai null, yang menandakan bahwa simpul tersebut belum tertaut.

2.2 Program InsertDLL

```
package pekan6_2511532001;

public class InsertDLL_2511532001 {
    //menambahkan node di awal DLL
    static NodeDLL_2511532001 insertBegin_2001(NodeDLL_2511532001
head_2001, int data_2001) {
        // buat node baru
        NodeDLL_2511532001 new_node_2001 = new
NodeDLL_2511532001(data_2001);
        // jadikan pointer nextnya head
        new_node_2001.next_2001 = head_2001;
        // jadikan pointer prev head ke new_node
        if (head_2001 != null) {
            head_2001.prev_2001 = new_node_2001;
        }
    }
}
```

```

    }
    return new_node_2001;
}
// fungsi menambahkan node di akhir
public static NodeDLL_2511532001 insertEnd_2001(NodeDLL_2511532001
head_2001, int newData_2001) {
    // buat node baru
    NodeDLL_2511532001 newNode_2001 = new
NodeDLL_2511532001(newData_2001);
    //jika dll null jadikan head
    if (head_2001 == null) {
        head_2001 = newNode_2001;
    }
    else {
        NodeDLL_2511532001 curr_2001 = head_2001;
        while (curr_2001.next_2001 != null) {
            curr_2001 = curr_2001.next_2001;
        }
        curr_2001.next_2001 = newNode_2001;
        newNode_2001.prev_2001 = curr_2001;
    }
    return head_2001;
}
// fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511532001
insertAtPosition_2001(NodeDLL_2511532001 head_2001, int pos_2001, int
new_data_2001) {
    // buat node baru
    NodeDLL_2511532001 new_node_2001 = new
NodeDLL_2511532001(new_data_2001);
    if (pos_2001 == 1) {
        new_node_2001.next_2001 = head_2001;
        if (head_2001 != null) {
            head_2001.prev_2001 = new_node_2001;
        }
        head_2001 = new_node_2001;
        return head_2001;
    }
    NodeDLL_2511532001 curr_2001 = head_2001;
    for (int i_2001 = 1; i_2001 < pos_2001 - 1 && curr_2001 != null;
++i_2001) {
        curr_2001 = curr_2001.next_2001;
    }
    if (curr_2001 == null) {
        System.out.println("Posisi tidak ada");
        return head_2001;
    }
    new_node_2001.prev_2001 = curr_2001;
    new_node_2001.next_2001 = curr_2001.next_2001;
    curr_2001.next_2001 = new_node_2001;
    if (new_node_2001.next_2001 != null) {
        new_node_2001.next_2001.prev_2001 = new_node_2001; }
    return head_2001;
}

```

```

    }
    public static void PrintList_2511532001(NodeDLL_2511532001 head_2001) {
        NodeDLL_2511532001 curr_2001 = head_2001;
        while (curr_2001 != null) {
            System.out.print(curr_2001.data_2001 + " <-> ");
            curr_2001 = curr_2001.next_2001;
        }
        System.out.println();
    }
    public static void main (String[] args) {
        NodeDLL_2511532001 head_2001 = new NodeDLL_2511532001(2);
        head_2001.next_2001 = new NodeDLL_2511532001(3);
        head_2001.next_2001.prev_2001 = head_2001;
        head_2001.next_2001.next_2001 = new NodeDLL_2511532001(5);
        head_2001.next_2001.next_2001.prev_2001 = head_2001.next_2001;
        // cetak DLL awal
        System.out.print("DLL awal: ");
        PrintList_2511532001 (head_2001);
        // tambah 1 di awal
        head_2001 = insertBegin_2001(head_2001, 1);
        System.out.print("Simpul 1 ditambah di awal: ");
        PrintList_2511532001 (head_2001);
        // tambah 6 di akhir
        System.out.print("Simpul 6 ditambah di akhir: ");
        int data_2001 = 6;
        head_2001 = insertEnd_2001(head_2001, data_2001);
        PrintList_2511532001 (head_2001);
        // menambah node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4: ");
        int data2_2001 = 4;
        int pos_2001 = 4;
        head_2001 = insertAtPosition_2001 (head_2001, pos_2001, data2_2001);
        PrintList_2511532001 (head_2001);
    }
}

```

InsertDLL mengimplementasikan fungsi penyisipan simpul dengan penautan pointer ganda. Pada metode insertBegin, ketika simpul baru disisipkan di depan, pointer next dari simpul baru akan diarahkan ke head lama, dan jika antrian tidak kosong, pointer prev dari head lama wajib di-update untuk menunjuk ke simpul baru tersebut sebelum status head dipindahkan.

Pada metode insertEnd, program melakukan penelusuran terlebih dahulu untuk menemukan simpul terakhir (tail), lalu menautkan next dari ekor tersebut ke simpul baru, sekaligus mengatur prev dari simpul baru agar menunjuk ke tail lama.

Adapun pada metode insertAtPosition, program menyisipkan simpul di tengah-tengah dengan cara mengamankan empat buah pointer sekaligus, menyambungkan next dan prev dari simpul baru ke simpul sekitarnya, serta memperbarui pointer next dari simpul sebelum dan pointer prev dari simpul sesudahnya.

2.3 Program PenelusuranDLL

```
package pekan6_2511532001;

public class PenelusuranDLL_2511532001 {
    //fungsi penelusuran maju
    static void forwardTraversal_2001(NodeDLL_2511532001 head_2001) {
        //mulai penelusuran dari head
        NodeDLL_2511532001 curr_2001 = head_2001;
        //lanjutkan sampai akhir
        while (curr_2001 != null) {
            //print data
            System.out.print(curr_2001.data_2001 + " <-> ");
            //pindah ke node berikutnya
            curr_2001 = curr_2001.next_2001;
        }
        //print spasi
        System.out.println();
    }

    // fungsi penelusuran mundur
    static void backwardTraversal_2001(NodeDLL_2511532001 tail_2001) {
        //mulai dari akhir
        NodeDLL_2511532001 curr_2001 = tail_2001;
        //lanjut sampai head
        while (curr_2001 != null) {
            //cetak data
            System.out.print(curr_2001.data_2001 + " <-> ");
            // pindah ke node sebelumnya
            curr_2001 = curr_2001.prev_2001;
        }
        // cetak spasi
        System.out.println();
    }

    public static void main (String[] args) {
        // cetak DLL
        NodeDLL_2511532001 head_2001 = new NodeDLL_2511532001(1);
        NodeDLL_2511532001 second_2001 = new NodeDLL_2511532001(2);
    }
}
```



```

NodeDLL_2511532001 third_2001 = new NodeDLL_2511532001(3);

head_2001.next_2001 = second_2001;
second_2001.prev_2001 = head_2001;
second_2001.next_2001 = third_2001;
third_2001.prev_2001 = second_2001;

System.out.println("Penelusuran maju: ");
forwardTraversal_2001(head_2001);

System.out.println("Penelusuran mundur: ");
backwardTraversal_2001(third_2001);

}
}

```

Program PenelusuranDLL mampu mendemonstrasikan keunggulan utama DLL melalui fungsi pembacaan data secara dua arah (bidirectional traversal) berkat pointer ganda. Fungsi forwardTraversal bekerja identik dengan SLL, yakni memanfaatkan while loop untuk bergeser membaca data dari depan (head) ke belakang dengan mengikuti jejak pointer next.

Di sisi lain, fungsi backwardTraversal memanfaatkan pointer prev untuk memulai pembacaan dari simpul terakhir (tail) dan secara sistematis menelusuri mundur jejak pointer tersebut hingga mencapai node terdepan yang bernilai null.

2.4 Program HapusDLL

```

package pekan6_2511532001;

public class HapusDLL_2511532001 {
    // fungsi menghapus node awal
    public static NodeDLL_2511532001 delHead_2001(NodeDLL_2511532001
head_2001) {
        if (head_2001 == null) {
            return null;
        }
        NodeDLL_2511532001 temp_2001 = head_2001;
        head_2001 = head_2001.next_2001;
        if (head_2001 != null) {
            head_2001.prev_2001 = null;
        }
        return head_2001;
    }

    // fungsi menghapus di akhir

```

```

    public static NodeDLL_2511532001 delLast_2001(NodeDLL_2511532001
head_2001) {
    if (head_2001 == null) {
        return null;
    }
    if (head_2001.next_2001 == null) {
        return null;
    }
    NodeDLL_2511532001 curr_2001 = head_2001;
    while (curr_2001.next_2001 != null) {
        curr_2001 = curr_2001.next_2001;
    }
    // update pointer previous node
    if (curr_2001.prev_2001 != null) {
        curr_2001.prev_2001.next_2001 = null;
    }
    return head_2001;
}

//fungsi menghapus node posisi tertentu
    public static NodeDLL_2511532001 delPos_2001(NodeDLL_2511532001
head_2001, int pos_2001) {
    // jika DLL kosong
    if (head_2001 == null) {
        return head_2001;
    }
    NodeDLL_2511532001 curr_2001 = head_2001;
    // telusuri sampai ke node yang akan dihapus
    for (int i_2001 = 1; curr_2001 != null && i_2001 < pos_2001;
++i_2001) {
        curr_2001 = curr_2001.next_2001;
    }
    // jika posisi tidak ditemukan
    if (curr_2001 == null) {
        return head_2001;
    }
    // Update pointer
    if (curr_2001.prev_2001 != null) {
        curr_2001.prev_2001.next_2001 = curr_2001.next_2001;
    }
    if (curr_2001.next_2001 != null) {
        curr_2001.next_2001.prev_2001 = curr_2001.prev_2001;
    }
    //jika yang dihapus head
    if (head_2001 == curr_2001) {
        head_2001 = curr_2001.next_2001;
    }
    return head_2001;
}

// fungsi mencetak DLL
    public static void printList_2001(NodeDLL_2511532001 head_2001) {
        NodeDLL_2511532001 curr_2001 = head_2001;
        while (curr_2001 != null) {

```

```

        System.out.print(curr_2001.data_2001 + " <-> ");
        curr_2001 = curr_2001.next_2001;
    }
    System.out.println();
}
public static void main(String[] args) {
    // buat sebuah DLL
    NodeDLL_2511532001 head_2001 = new NodeDLL_2511532001(1);
    head_2001.next_2001 = new NodeDLL_2511532001(2);
    head_2001.next_2001.prev_2001 = head_2001;
    head_2001.next_2001.next_2001 = new NodeDLL_2511532001(3);
    head_2001.next_2001.next_2001.prev_2001 = head_2001.next_2001;
    head_2001.next_2001.next_2001.next_2001 = new
NodeDLL_2511532001(4);
    head_2001.next_2001.next_2001.next_2001.prev_2001 =
head_2001.next_2001.next_2001;
    head_2001.next_2001.next_2001.next_2001.next_2001 = new
NodeDLL_2511532001(5);
    head_2001.next_2001.next_2001.next_2001.next_2001.prev_2001 =
head_2001.next_2001.next_2001.next_2001;

    System.out.print("DLL Awal: ");
    printList_2001(head_2001);

    System.out.print("Setelah head dihapus: ");
    head_2001 = delHead_2001(head_2001);
    printList_2001(head_2001);

    System.out.print("Setelah node terakhir dihapus: ");
    head_2001 = dellast_2001(head_2001);
    printList_2001(head_2001);

    System.out.print("menghapus node ke 2: ");
    head_2001 = delPos_2001(head_2001, 2);

    printList_2001(head_2001);
}
}

```

Program HapusDLL memperlihatkan proses penghapusan simpul yang lebih kompleks, mengingat ada dua arah pointer yang harus diamankan sebelum sebuah node dilepaskan.

Saat mengeksekusi delHead, program akan memindahkan referensi head ke simpul di sebelahnya, lalu "memutuskan" pointer prev dari head yang baru menjadi null agar referensi ke simpul lama benar-benar hilang.

Pada `delLast`, program menelusuri rantai untuk menemukan simpul terakhir, lalu menghapus akses ke simpul tersebut dengan cara mengubah pointer `next` pada simpul sebelumnya (`second last`) menjadi `null`.

Sementara pada `delPos` (penghapusan di tengah), program akan mengekstrak simpul target dari barisan dengan cara menghubungkan pointer `next` pada simpul sebelumnya secara langsung dengan pointer `prev` pada simpul sesudahnya, seolah-olah "menjahit" kembali barisan yang putus tanpa menghiraukan simpul target yang telah dibuang.

BAB 3

TUGAS

1.1 Program Lagu

```

2. package pekan6_2511532001;
3.
4. public class Lagu_2511532001 {
5.
6.     private String judul_2001;
7.     private String penyanyi_2001;
8.     private Lagu_2511532001 next_2001;
9.     private Lagu_2511532001 prev_2001;
10.
11.     // konstruktor
12.     public Lagu_2511532001(String judul_2001, String penyanyi_2001) {
13.         this.judul_2001 = judul_2001;
14.         this.penyanyi_2001 = penyanyi_2001;
15.         this.next_2001 = null;
16.         this.prev_2001 = null;
17.     }
18.
19.     // getter dan setter
20.     public String getJudul_2001() { return judul_2001; }
21.     public void setJudul_2001(String judul_2001) { this.judul_2001 =
judul_2001; }
22.
23.     public String getPenyanyi_2001() { return penyanyi_2001; }
24.     public void setPenyanyi_2001(String penyanyi_2001) {
this.penyanyi_2001 = penyanyi_2001; }
25.
26.     public Lagu_2511532001 getNext_2001() { return next_2001; }
27.     public void setNext_2001(Lagu_2511532001 next_2001) {
this.next_2001 = next_2001; }
28.
29.     public Lagu_2511532001 getPrev_2001() { return prev_2001; }
30.     public void setPrev_2001(Lagu_2511532001 prev_2001) {
this.prev_2001 = prev_2001; }
31. }

```

Sistem playlist musik dikembangkan dengan memanfaatkan kerangka Double Linked List. Entitas terkecil dari struktur ini dibungkus di dalam kelas Lagu yang menyimpan atribut berupa judul dan penyanyi. Class ini mengimplementasikan pointer memori ganda, yaitu next untuk menyimpan alamat memori dari lagu yang ada di urutan berikutnya, serta prev untuk melacak alamat memori dari lagu sebelumnya. Melalui constructor, kedua pointer ini diinisialisasi sebagai null.

3.2 Program Musik

```

package pekan6_2511532001;
import java.util.Scanner;

public class Musik_2511532001 {
    private Lagu_2511532001 head_2001;
    private Lagu_2511532001 tail_2001; // Pointer tambahan untuk memudahkan
    tampil mundur

    public Musik_2511532001() {
        this.head_2001 = null;
        this.tail_2001 = null;
    }

    // 1. tambah lagu di akhir playlist
    public void tambahLagu_2001(String judul_2001, String penyanyi_2001) {
        Lagu_2511532001 nodeBaru_2001 = new Lagu_2511532001(judul_2001,
        penyanyi_2001);

        if (head_2001 == null) {
            head_2001 = nodeBaru_2001;
            tail_2001 = nodeBaru_2001;
        } else {
            // menyambungkan ekor ke node baru tanpa harus traversal dari
            head
            tail_2001.setNext_2001(nodeBaru_2001);
            nodeBaru_2001.setPrev_2001(tail_2001);
            tail_2001 = nodeBaru_2001; // pindah status ekor
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

    // 2. hapus lagu pertama (head)
    public void hapusLaguAwal_2001() {
        if (head_2001 == null) {
            System.out.println("Playlist kosong, tidak ada lagu yang bisa
            dihapus.");
            return;
        }
        System.out.println("Menghapus lagu: " + head_2001.getJudul_2001());
        head_2001 = head_2001.getNext_2001(); // geser head ke node kedua

        if (head_2001 != null) {
            head_2001.setPrev_2001(null); // putus memori prev node kedua
            ke node lama
        } else {
            tail_2001 = null; // jika lagu habis, tail null
        }
    }

    // 3. tampilkan semua lagu dari awal ke akhir

```

```

public void tampilMaju_2001() {
    if (head_2001 == null) {
        System.out.println("Playlist kosong.");
        return;
    }
    Lagu_2511532001 temp_2001 = head_2001;
    int no_2001 = 1;
    System.out.println("\n=== Playlist Musik (Maju) ===");
    while (temp_2001 != null) {
        System.out.println(no_2001 + ". " + temp_2001.getJudul_2001() +
" - " + temp_2001.getPenyanyi_2001());
        temp_2001 = temp_2001.getNext_2001();
        no_2001++;
    }
}

// 4. tampilkan semua lagu dari akhir ke awal (Backward Traversal)
public void tampilMundur_2001() {
    if (tail_2001 == null) {
        System.out.println("Playlist kosong.");
        return;
    }
    Lagu_2511532001 temp_2001 = tail_2001; // mulai dari tail
    int no_2001 = 1;
    System.out.println("\n=== Playlist Musik (Mundur) ===");
    while (temp_2001 != null) {
        System.out.println(no_2001 + ". " + temp_2001.getJudul_2001() +
" - " + temp_2001.getPenyanyi_2001());
        temp_2001 = temp_2001.getPrev_2001(); // mundur menggunakan
pointer prev
        no_2001++;
    }
}

// 5. cari lagu berdasarkan judul (case insensitive)
public void cariLagu_2001(String cariJudul_2001) {
    if (head_2001 == null) {
        System.out.println("Playlist kosong.");
        return;
    }
    Lagu_2511532001 temp_2001 = head_2001;
    boolean ditemukan_2001 = false;
    int posisi_2001 = 1;

    while (temp_2001 != null) {
        if (temp_2001.getJudul_2001().equalsIgnoreCase(cariJudul_2001))
        {
            System.out.println("Lagu ditemukan di urutan ke-" +
posisi_2001 + " -> " + temp_2001.getJudul_2001() + " - " +
temp_2001.getPenyanyi_2001());
            ditemukan_2001 = true;
            break;
        }
    }
}

```

```

        temp_2001 = temp_2001.getNext_2001();
        posisi_2001++;
    }

    if (!ditemukan_2001) {
        System.out.println("Lagu dengan judul '" + cariJudul_2001 + "'
tidak ditemukan.");
    }
}

// main menu Program
public static void main(String[] args) {
    Scanner sc_2001 = new Scanner(System.in);
    Musik_2511532001 spotify_2001 = new Musik_2511532001();
    int pilihan_2001;

    do {
        System.out.println("\nPlaylist Musik NIM: 2511532001");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2001 = sc_2001.nextInt();
        sc_2001.nextLine(); // membersihkan buffer enter

        switch (pilihan_2001) {
            case 1:
                System.out.print("Judul: ");
                String judul_2001 = sc_2001.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi_2001 = sc_2001.nextLine();
                spotify_2001.tambahLagu_2001(judul_2001,
penyanyi_2001);
                break;
            case 2:
                spotify_2001.hapusLaguAwal_2001();
                break;
            case 3:
                spotify_2001.tampilMaju_2001();
                break;
            case 4:
                spotify_2001.tampilMundur_2001();
                break;
            case 5:
                System.out.print("Cari Judul: ");
                String cari_2001 = sc_2001.nextLine();
                spotify_2001.cariLagu_2001(cari_2001);
                break;
            case 6:
                System.out.println("Program selesai.");

```



```

        break;
    default:
        System.out.println("Pilihan tidak valid.");
    }
} while (pilihan_2001 != 6);
sc_2001.close();
}
}

```

Class Musik berperan sebagai pusat pengendalian logika program yang mengintegrasikan seluruh operasi utama melalui antarmuka Switch-Case. Operasi Tambah Lagu dilakukan dengan menyisipkan node baru di akhir senarai menggunakan referensi tail, sehingga proses menjadi lebih efisien karena tidak perlu menelusuri node dari awal. Pada operasi Hapus Lagu Pertama, program menerapkan konsep Delete Head dengan memindahkan head ke node berikutnya dan mengatur pointer prev dari head baru menjadi null. Selain itu, program menyediakan fitur Tampil Maju dan Tampil Mundur untuk menelusuri data dari dua arah, yaitu dari head menggunakan pointer next, serta dari tail menggunakan pointer prev. Operasi Cari Lagu dilakukan secara linier dengan metode equalsIgnoreCase, sehingga pencarian tidak membedakan huruf besar dan kecil. Secara keseluruhan, program sudah menerapkan konsep Doubly Linked List dengan baik melalui penggunaan pointer head, tail, next, dan prev, serta dilengkapi validasi kondisi kosong agar operasi tetap aman saat senarai tidak memiliki data.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List (DLL) merupakan penyempurnaan dari Single Linked List yang memberikan fleksibilitas navigasi dua arah melalui penambahan atribut pointer prev. Modifikasi arsitektur memori ini memungkinkan operasi penelusuran data (traversal) dapat bergerak baik bergerak maju (forward) menuju pointer next yang bernilai null, maupun mundur (backward) menuju pointer prev yang bernilai null. Meskipun memberikan kemudahan, keunggulan ini harus dibayar dengan konsekuensi naiknya kompleksitas algoritma pada saat operasi penyisipan (insertion) maupun penghapusan (deletion).

DAFTAR PUSTAKA

- [1] W3Schools, "Java LinkedList". [Daring]. Tersedia pada:
https://www.w3schools.com/java/java_linkedlist.asp. [Diakses: 14 Mei 2026].